

Sistemas Operativos 2026



Clase 4 U2 Región crítica

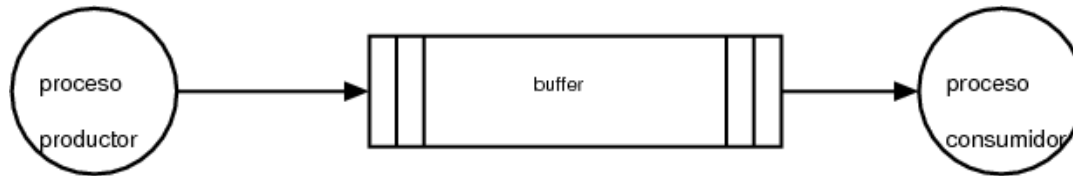
Ing. Juan Manuel Calvo

Comunicación entre procesos

- El modelo de procesos es una manera útil de aislar programas en ejecución
 - Separa recursos, estado, etc.
 - simplifica ampliamente muchos programas
 - no nos preocupamos por lo que están haciendo otros procesos
 - limitados canales de comunicación (wait, kill, etc)
- Desafortunadamente, algunas aplicaciones funcionan mejor si se le permite a varios hilos comunicarse y sincronizarse más fácilmente los unos con los otros
 - y esto puede complicar las cosas nuevamente.

Ejemplo: Problema del productor/consumidor

- Cada tanto el productor coloca un ítem en el buffer.
- Cada tanto el consumidor saca un ítem del buffer.
- se requiere una cuidadosa sincronización (ambos corren simultaneamente)
 - Si el buffer está vacío **el consumidor** debe esperar.
 - Si el buffer está lleno **el productor** debe esperar.
- La solución típica implica una variable compartida, la cuenta.
 - También conocido como el problema del buffer limitado.



¿Porqué sugen estas complicaciones?

- El modelo de procesos permite que más de un programa corra "al mismo tiempo".
 - No están corriendo realmente al mismo tiempo (En una máquina de un sólo procesador).
 - "Al mismo tiempo" significa un arbitrario -- e impredecible -- entrelazado de instrucciones de máquina de los procesos "simultáneos".
- Problema: las operaciones lógicas en datos compartidos frecuentemente involucran más de una instrucción de assembler.
 - Su proceso puede ser detenido por una interrupción, otro proceso podría correr y alterar los datos compartidos el medio de la operación.

Dos procesos "simultáneos" A y B



Instruccion A1

Instruccion B1

Instruccion A2

Instruccion B2

Instruccion A3

Instruccion B3

Instruccion A4

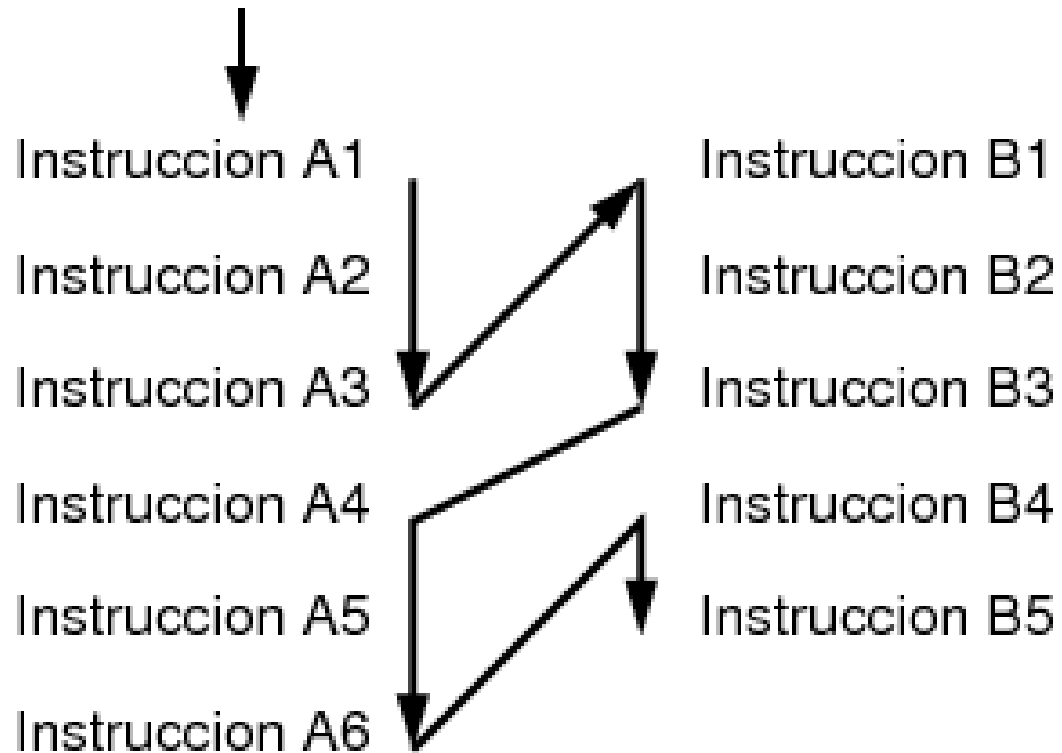
Instruccion B4

Instruccion A5

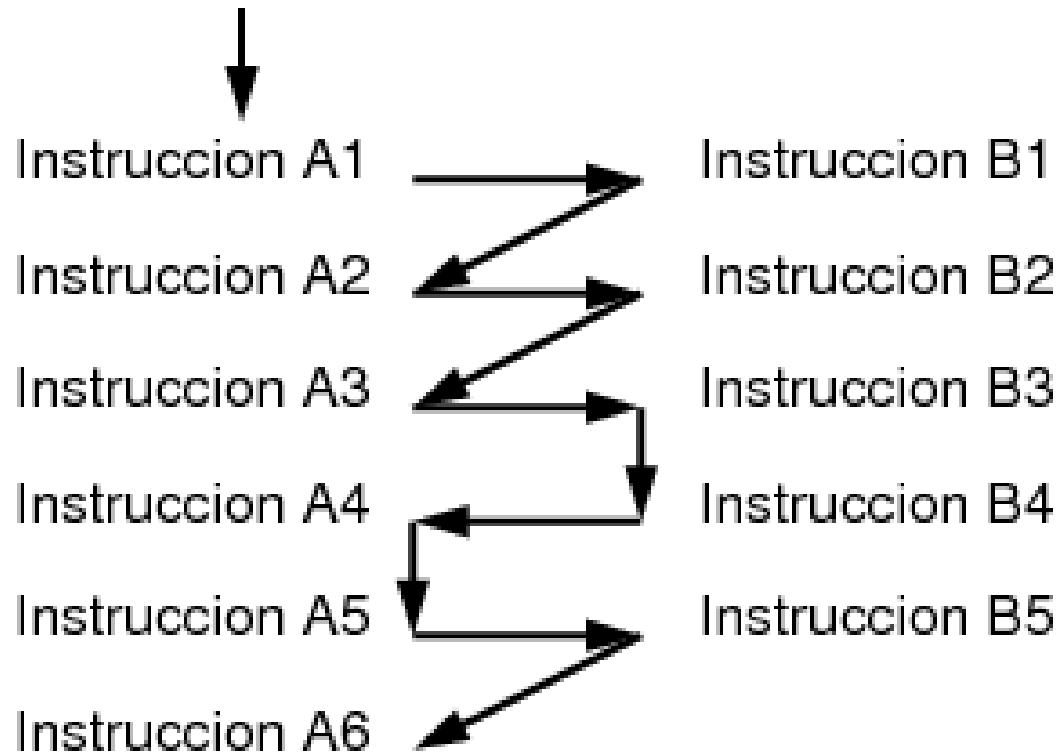
Instruccion B5

Instruccion A6

... podrian ser planificados así:



... o tal vez así:



Normalmente, esto no nos importa

- Después de todo los dos procesos están aislados
 - diferente memoria, registros, etc
 - no se puede distinguir entre las diferentes secuencias de planificación, por lo tanto no hay problema.
- ¿pero que pasa si dos hilos comparten el acceso a la misma memoria?
 - Aquí es donde los problemas comienzan.

Ejemplo de dos hilos y una variable compartida

Imaginemos que tenemos dos vendedores de tickets A y B, cada uno tiene una tarea:

```
while (TRUE) {  
    A_espera_compra();  
    vendidos += 1;  
}
```

```
while (TRUE) {  
    B_espera_compra();  
    vendidos += 1 ;  
}
```

Al compilar el código de alto nivel

- Vendedos += 1;

- Resulta:

- Hilo de A

load a0,vendidos

inc a0

store a0,vendidos

- Hilo de B

load a1,vendidos

inc a1

store a1,vendidos

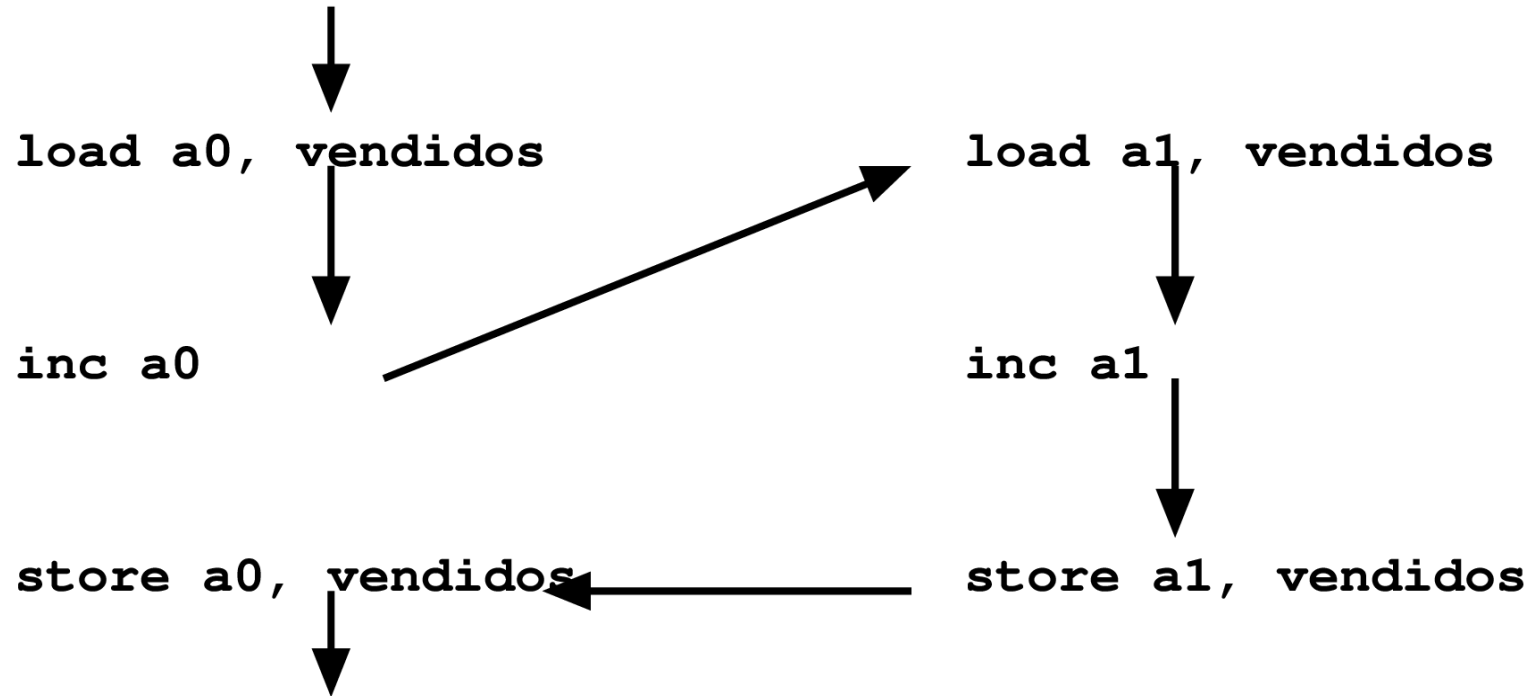
- a0 y a1 se usan de registro temporal para incrementar la cuenta de tickets vendidos. Notar que estamos siendo cuidadosos evitando usar el mismo registro



UNIVERSIDAD
DEL CEMA

UCEMA

¿Funciona esto si dos clientes hacen la compra casi al mismo tiempo?



Carreras y sincronización

- Las operaciones en estructuras de datos compartidos frecuentemente consisten en "ráfagas" cortas de instrucciones.
- Cuando dos procesos/hilos están ejecutándose concurrentemente el resultado puede depender del preciso entrelazado de los dos flujos de instrucciones
 - esto es lo que se llama una condición de **carrera**.
 - ¡las carreras causan bugs que son difíciles de reproducir!
- Además de la carreras, otra cuestión es la sincronización (un proceso/hilo puede estar esperando por resultados de un cálculo de otro).
 - En la espera ¿podemos evitar la espera activa?

Condición de carrera



- Comportamiento indeseable que puede ocurrir cuando el resultado depende del orden de las operaciones

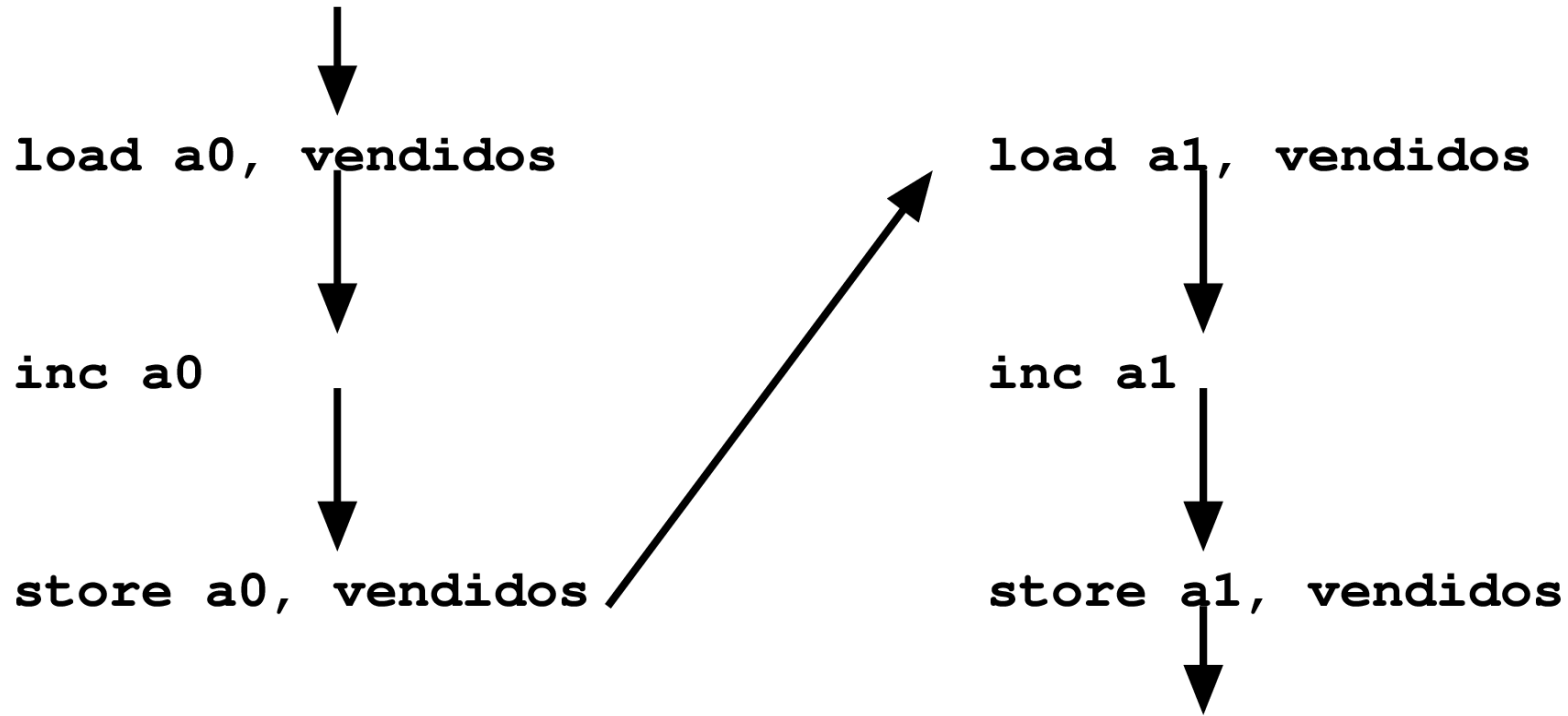
Las carreras son malas

- Pueden causar
 - Impredictibilidad en los resultados
 - Pérdida de datos
 - Problemas de seguridad
 - Que el sistema se cuelgue
 - Que se produzca un reset del sistema

Preguntas razonables para hacerse en este momento...

- Al final de este entrelazado, ¿en cuanto se incrementa vendidos?
 - ¿Hay otros entrelazados "incorrectos" de estas instrucciones?
 - ¿hay algún entrelazado "correcto"?
 - si es así, ¿como puedo asegurarme que ocurra?

Un entrelazado alternativo:



¿Qué está pasando?

- Se espera que estas instrucciones se ejecuten juntas:

```
load a0, vendidos
```

```
increment a0
```

```
store a0, vendidos
```

- Otros hilos que operan sobre **esta variable no deben** interrumpir esta operación.
- Nos gustaría agrupar esas tres instrucciones juntas, de tal manera que otros procesos/hilos no interfieran con ellas.
- Este es el problema de la ***exclusión mutua***.

En otras palabras...

- comienza la región crítica (a partir de acá nadie debe interrumpirla)

```
load a0, vendidos
```

```
inc a0
```

```
store a0, vendidos
```

- termina la región crítica (nuevamente se aceptan interrupciones).

Exclusión mutua

- Necesitamos asegurar la ejecución "atómica" de una **secuencia** de instrucciones
 - Por lo menos en los que respecta a otros hilos que accedan a los datos.
- ¿cómo podemos hacer esto?
 - Una interrupción puede ocurrir en cualquier momento y darle el control a otro hilo.

Abstracción de la región crítica

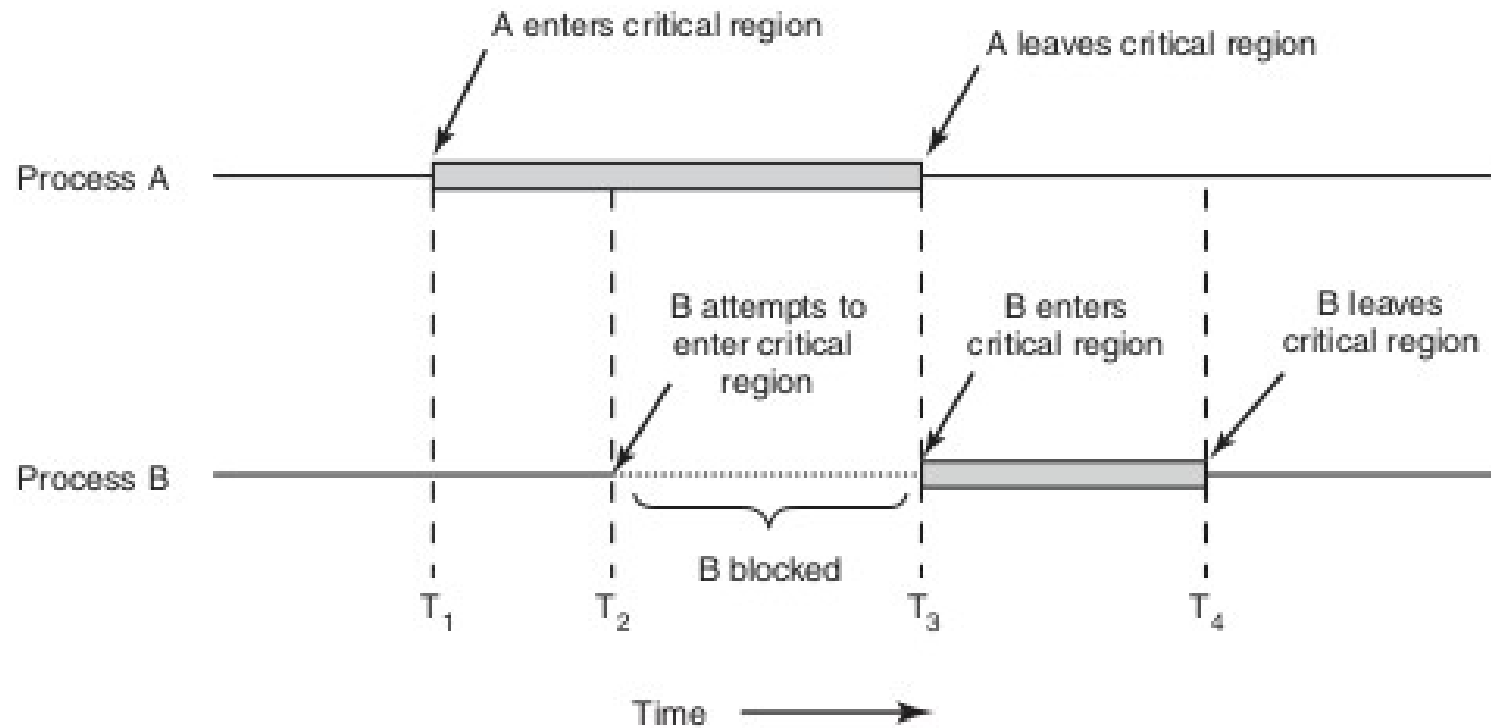
- Como un "vigilante" protegiendo regiones de código:
 - COMENZAR LA REGIÓN CRITICA
 - si no hay otro en la región crítica, puedo entrar.
 - de lo contrario espero
 - Hacer el trabajo de la región crítica
 - FINALIZAR LA REGIÓN CRITICA
 - Renunciar al acceso exclusivo, permitir a otros ingresar.
- Condición necesaria:
 - Asumimos que todos los hilos estan respetando las mismas reglas.

Regiones críticas



- La parte de un programa que utiliza una variable compartida se denomina **región crítica**.
- Dos (o más) programas no deben estar en sus regiones críticas al mismo tiempo.
- Un programa (o proceso, hilo o dispositivo) debe bloquearse si necesita acceder a la región crítica mientras el otro se encuentra en la suya.

Abstracción de la región crítica II



Región Crítica: Requisitos para una buena solución



Una buena solución debe satisfacer estos tres requisitos:

- 1) Exclusión mutua: no debe ser posible que dos procesos estén simultáneamente dentro de sus regiones críticas.
- 2) Condición de progreso: no debe haber un estado donde todos están esperando que algún otro haga algo antes que alguno pueda entrar.
- 3) Condición de espera limitada: ningún proceso debería tener que esperar indefinidamente para entrar en su región crítica.

En ningún caso la solución puede depender de suposiciones sobre las velocidades o el número de CPUs.

Exclusión mutua con espera activa

- Deshabilitar interrupciones
- Variables de lock
- Alternancia estricta
- Solución de Peterson
- Instrucciones atómicas/barreras
 - TSL (Test and Set Lock)
 - CAS (Compare and Swap)

Deshabilitar las interrupciones

- Trabaja impidiendo la planificación preventiva por el reloj u otra interrupción.
 - apagar las interrupciones antes de entrar en la región crítica y encenderlas al finalizar.
- Se usa en las partes de bajo nivel del sistema operativo (ej: durante el manejo de interrupciones).
- Satisface los requerimientos, pero no es adecuada para los procesos de usuario
 - Requiere cuidado: si nos olvidamos de restablecerlas va a ser necesario rebootear.
 - Demasiado poderosa: previene que todos los otros procesos puedan ser reemplazados, no solamente aquellos que están entrando en una región crítica.
 - Inflexible: todo o nada.
- ATENCIÓN: En sistemas con más de una CPU deshabilitando las interrupciones en una no impide que otra CPU acceda a datos compartidos.

Variables de locks I

- ¿Qué les parece esto?

```
while (lock != 0);  
/* Start critical region */  
lock = 1;  
  
...  
/* End critical region */  
lock = 0;
```

Variables de locks II



```
while (lock != 0);  
/* Start critical region */  
lock = 1;  
  
...  
  
/* End critical region */  
lock = 0;
```

<---- Interrupción

- No funciona: hay una ventana entre la comprobación de cero y su puesta a 1.

Y esta otra?

```
while( lock++ != 0 ) lock--;  
    /* critical region ...*/
```

- ¿Es lock++ atómico?

Veamos el assembler

- `while( lock++ != 0 ) lock--;` en código gcc X86:

`tstlock:`

```
mov    eax, lock      ; se guarda en ax el valor de lock sin incrementar
lea    edx, [rax+1]    ; sumo 1 a lock
mov    lock, edx       ; lo guardo en memoria
test   eax, eax        ; ¿lock era 0 ?
je     .entercr        ; voy a la región crítica
mov    eax, lock       ;
sub    eax, 1          ; decremento a lock
mov    lock, eax       ; lo guardo en memoria
jmp    .tstlock        ; repito el ciclo
```

`.entercr:`

- Para una variable en memoria ¡El incremento no es atómico!.



UNIVERSIDAD
DEL CEMA

UCEMA

Alternancia estricta

Process 0

```
while (TRUE) {  
    while (turn != 0);  
    critical_region();  
    turn = 1;  
    noncritical_region();  
}
```

Process 1

```
while (TRUE) {  
    while (turn != 1);  
    critical_region();  
    turn = 0;  
    noncritical_region();  
}
```

- Variable turn --> Determina quien puede entrar en la región crítica
- Comprobar continuamente una variable hasta que tenga un valor determinado se denomina espera activa (busy waiting).

Desventajas de la alternancia estricta

- Supongamos que el código de la región no crítica del Proceso 1 tarda mucho en ejecutarse.
- No volverá a entrar en su región crítica.
- Hasta que lo haga, el Proceso 0 no puede volver a entrar en su propia región crítica.
- Esto viola nuestro principio de eficiencia.



Algoritmo de Peterson (1981)

- Algoritmo simple que utiliza únicamente instrucciones ordinarias.



Algoritmo de Peterson

```
int turn, interested[2];
void enter_region(int process)
{
    int other;
    other = 1-process;
    interested[process] = TRUE;
    turn = process;
    while (turn == process && interested[other] == TRUE);
}
void leave_region(int process)
{
    interested[process] = FALSE;
}
```



Estas "soluciones" no funcionan

- La alternancia estricta y la solución de Peterson no funcionan en los procesadores modernos porque el hardware actual reordena las operaciones de lectura y escritura para mejorar el rendimiento, destruyendo el orden secuencial que estos algoritmos necesitan para funcionar.
- Los algoritmos clásicos de exclusión mutua nacieron bajo el concepto de consistencia secuencial. Esto significa que el código se ejecuta de manera estricta, línea por línea, en el orden exacto que escribió el programador. Los procesadores actuales para ser más rápidos rompen esta regla .

Razones de la falla

- Ejecución fuera de orden: El procesador analiza el código y cambia de lugar las instrucciones de lectura y escritura si detecta que no están relacionadas.
- Optimización de los compiladores: Los compiladores modernos también alteran el orden de las variables enmascarando el comportamiento real que el algoritmo de Peterson espera.
- Búferes de escritura (Store Buffers): Cada procesador guarda los cambios en su caché antes de enviarlos a la memoria RAM compartida. Otro procesador puede ver las modificaciones con retraso o en un orden distinto al del código.

La solución actual

- Para lograr sincronización segura en la actualidad, se utilizan barreras de memoria (memory barriers) o instrucciones a nivel de hardware diseñadas específicamente para detener temporalmente el reordenamiento y asegurar que un cambio sea visible para todos los núcleos de forma inmediata.

Instrucciones Atómicas



Son operaciones especiales que el procesador ejecuta en un solo paso indivisible. Ningún otro núcleo puede interferir o ver el estado a la mitad del proceso.

- **Mecanismo:** Bloquean temporalmente el acceso a una dirección de memoria específica (o su línea de caché) a nivel de hardware.
- **Operaciones:** Compare-And-Swap (CAS), Test-And-Set.
- **Cómo funciona:** El procesador lee un valor, lo compara con un valor esperado y, si coinciden, escribe el nuevo valor. Todo esto ocurre como una única acción "atómica".
- **Uso:** Es la base para construir los mutex en lenguajes como C++, Java o Rust.

Barreras de Memoria (Memory Barriers / Fences)

Son instrucciones de hardware que imponen un orden estricto en las operaciones de memoria, impidiendo que el procesador reordene las líneas de código a su antojo.

- **Mecanismo:** Actúan como una pared invisible en la línea de ejecución del procesador.
- **Regla:** Ninguna operación de lectura o escritura que esté después de la barrera puede ejecutarse antes que las operaciones que estaban antes de la barrera.

Barreras de Memoria (Memory Barriers / Fences)



Tipos principales:

- **Load Fence:** Ordena las lecturas de memoria.
- **Store Fence:** Vacía los store buffers hacia la caché para que otros núcleos vean los cambios inmediatamente.
- **Full Fence:** Ordena tanto lecturas como escrituras.
- **Uso real:** Si quisieras hacer funcionar el algoritmo de Peterson hoy, tendrías que colocar manualmente barreras de memoria entre la activación de la bandera (flag) y la lectura del turno.

Usar el hardware (TSL)

- Es mucho mas mas simple cuando tenemos soporte de la CPU
 - La lectura y la escritura en una sola operación atómica.
 - Muchas CPU modernas tienen algo así.
- Test-and-set-lock: TSL Rn, lock
 - Se transfiere a Rn el contenido de lock.
 - La memoria lock se carga con "1"
 - El bus de memoria se bloquea durante esta instrucción, otras CPUs no puede interferir.
- Otras instrucciones atómicas con la misma funcionalidad:
 - intercambiar (swap)

TSL

- Entrada a la región crítica

LOCK TSL R0, lock	Copiar lock a R0 y setearlo en 1
CMP R0, #0	¿era 0?
BNZ LOCK	saltar si no era 0

- Salida de la región crítica

MOVE lock, #0

En lugar de una instrucción especial, X86 tiene un prefijo LOCK que puede aplicarse a varias instrucciones.

Funciones TSL o similares

- Gcc Built-in functions for atomic memory access
- Windows
 - The Interlocked API
 - InterlockedIncrement64 function (winnt.h)

Spinlock

- Las tres soluciones correctas (Test and Set, alternancia estricta y algoritmo de Peterson) implican espera activa.
- También conocido como spinlock.
 - malo: esto puede ser ineficiente, los hilos bloqueados aún necesitan consumir CPU mientras están esperando.
 - peor: puede haber un bloqueo mutuo si el proceso que está esperando tiene prioridad más alta en el planificador.
 - Son útiles para esperas cortas, en los kernels de los SO.
- Para uso general, vamos a preferir una solución **bloqueante**.